MACHINE LEARNING OPERATIONS / ASSIGNMENT 01

End-to-End MLOps Pipeline for Heart Disease Risk Prediction

Reproducible modelling · experiment tracking · secure CI/CD · AWS EKS · observability

STUDENT	ASSIGNMENT	SUBMISSION
Rengaprakash S | 2025AE05107	AIMLCZG523 · BITS Pilani	[Confirm submission date]

Executive overview

This project develops a binary heart disease classifier using the UCI Heart Disease dataset (303 records), packages the fitted scikit-learn preprocessing and model pipeline, and serves predictions through FastAPI. GitHub Actions automates data checks, training, quality and security verification, image delivery to Amazon ECR, approval-gated deployment to Amazon EKS, and OWASP ZAP testing. Prometheus and Grafana provide monitoring.

SELECTED MODEL	TEST ACCURACY	TEST RECALL	TEST ROC-AUC
Logistic Regression	88.5%	92.9%	0.966

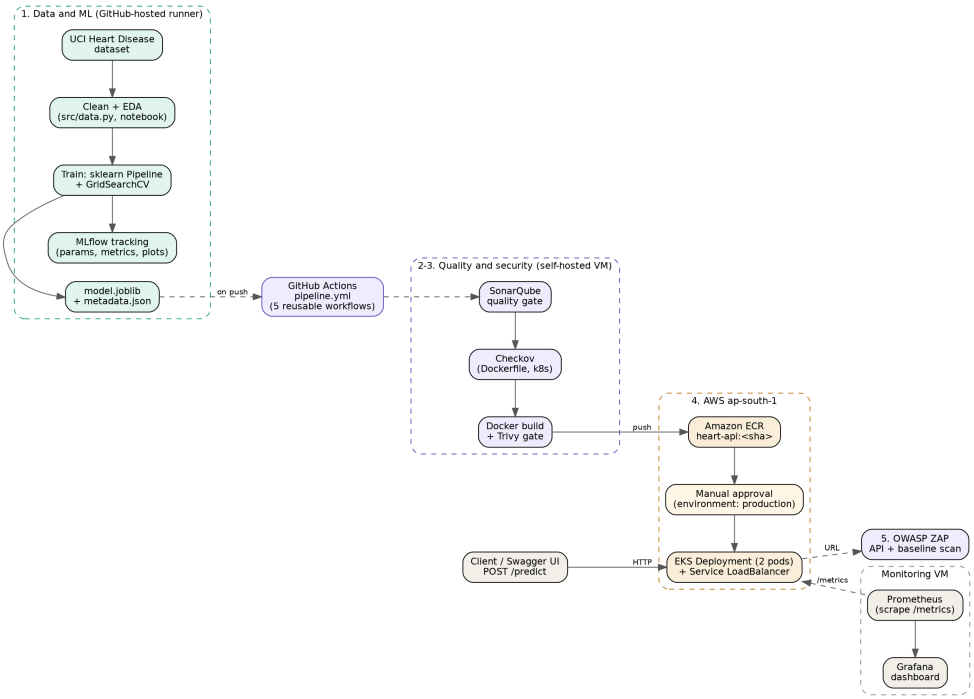


Figure 1. Source architecture: dataset → MLflow → CI/security → ECR/EKS → monitoring.

Repository: https://github.com/Rengaprakash4440/AIMLCZG523_Assignment_01

Demonstration video: [INSERT VERIFIED VIDEO URL BEFORE SUBMISSION]

1. Data acquisition and exploratory analysis

Data source: UCI Heart Disease (dataset ID 45). src/download_data.py retrieves the source data; src/data.py converts the original 0–4 target into binary labels (0 = absence; 1 = presence for 1–4), removes duplicates, and writes the cleaned dataset. The final data contains 303 records, 13 predictors (five numerical and eight categorical) and one binary target. Missing values occur in ca (four rows) and thal (two rows); imputation is intentionally deferred to the training pipeline to avoid validation leakage.

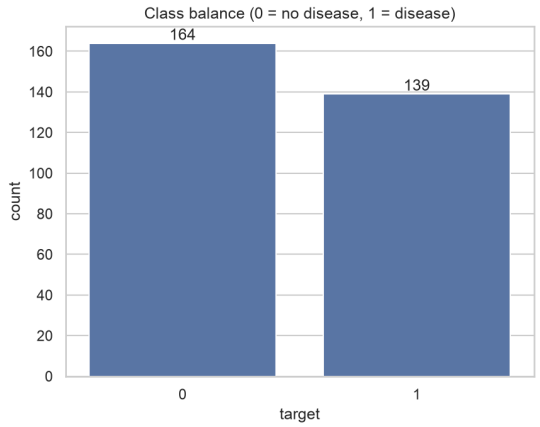


Figure 2. Class balance: 164 negative / 139 positive.

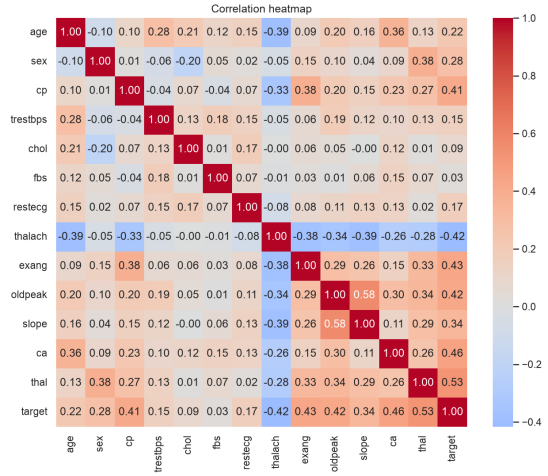


Figure 3. Correlation heatmap from EDA.

Exploratory findings

- Class ratio: 54.1% no disease versus 45.9% disease, sufficiently balanced for stratified evaluation without resampling.
- The strongest recorded associations were thal (r = 0.53), ca (0.46), exang (0.43), oldpeak (0.42), thalach (−0.42) and cp (0.41).
- Asymptomatic chest-pain category (cp = 4) showed about 73% disease prevalence in this sample; observed prevalence is descriptive, not causal.
- Four cholesterol observations above 400 mg/dL were retained because of the limited dataset size; preprocessing handles scale differences.

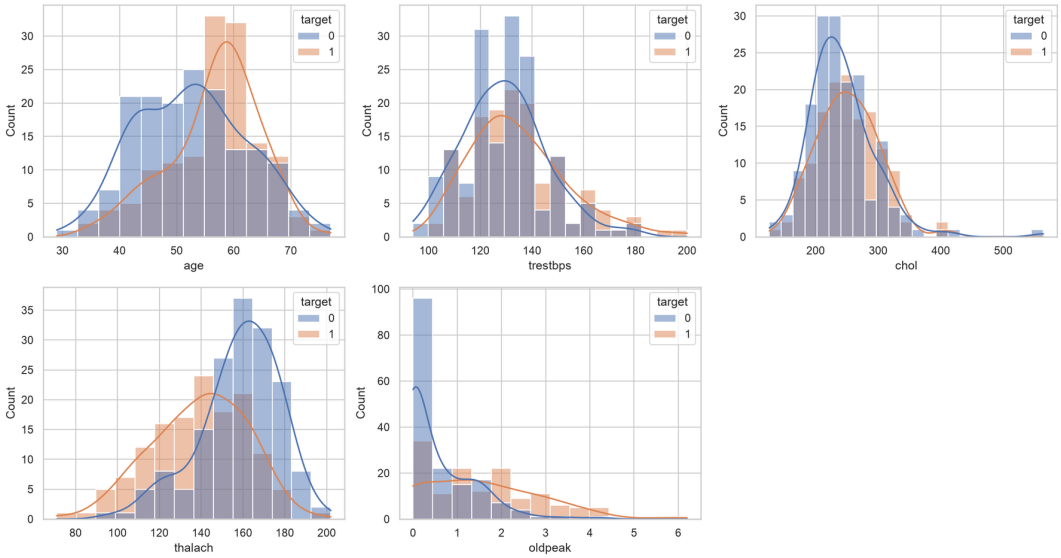


Figure 4. Numerical feature distributions stratified by observed outcome.

Reproduction: python -m src.download_data followed by data cleaning; figures and notebook are stored under report/figures/ and report/notebooks/01_eda.ipynb.

2. Feature engineering and model development

A single scikit-learn Pipeline couples a ColumnTransformer with the estimator. Numeric predictors use median imputation and standard scaling; categorical predictors use most-frequent imputation and OneHotEncoder(handle_unknown="ignore"). Fitting transformations inside cross-validation prevents test/validation statistics from leaking into model selection.

The 303 records were split into stratified 80% training (242) and 20% held-out testing (61) sets using seed 42. GridSearchCV tuned Logistic Regression and Random Forest using five-fold stratified cross-validation with ROC-AUC for refitting. The held-out test set was evaluated for both candidates to report performance; it was not used by the automated final-model selection rule.

Metric	Logistic Regression	Random Forest
CV ROC-AUC	0.902493	0.901952
Test accuracy	0.885	0.869
Test precision	0.839	0.833
Test recall	0.929	0.893
Test F1	0.881	0.862
Test ROC-AUC	0.966	0.948

Model selection methodology and limitation. The training script (src/train.py) selects the final model using the highest unrounded mean ROC-AUC from five-fold stratified cross-validation. Logistic Regression (C = 1) achieved 0.902493 versus 0.901952 for the best Random Forest (100 trees, maximum depth 5, minimum leaf size 3). Both values round to 0.902 at three decimal places, but this was not an exact tie. Logistic Regression was therefore selected by the implemented cross-validation score comparison; simpler inference and interpretability are additional practical benefits, not programmed tie-breakers. Test accuracy, recall and ROC-AUC were recorded for both candidates on the 61-record held-out split but were not inputs to the automated selection condition. Because the cross-validation difference is very small and the test set contains only 61 cases, performance estimates remain uncertain; a new independent evaluation would strengthen confidence. The precise CV values are recorded in models/gridsearch_*.csv.

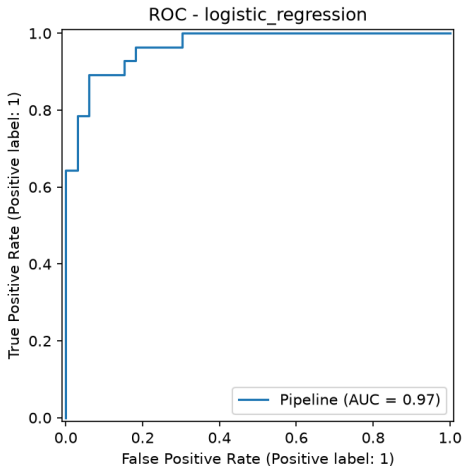


Figure 5. Logistic Regression ROC curve.

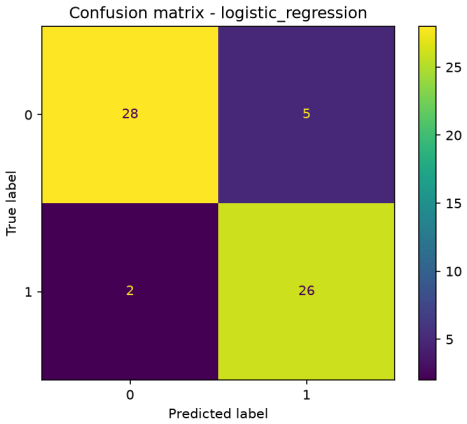


Figure 6. Logistic Regression confusion matrix.

Interpretation caution: the API returns predicted-class probability as “confidence”; probability calibration was not established by a calibration study. It is not an estimated clinical risk score.

3. Experiment tracking and reproducibility

MLflow logs one experiment run per model in the heart-disease experiment (SQLite backend). Parameters include model type, cross-validation folds, seed, test ratio and tuned hyperparameters; metrics include cross-validation and test accuracy, precision, recall, F1 and ROC-AUC. Logged artifacts include ROC curves, confusion matrices, grid-search CSVs and the cleaned dataset snapshot.

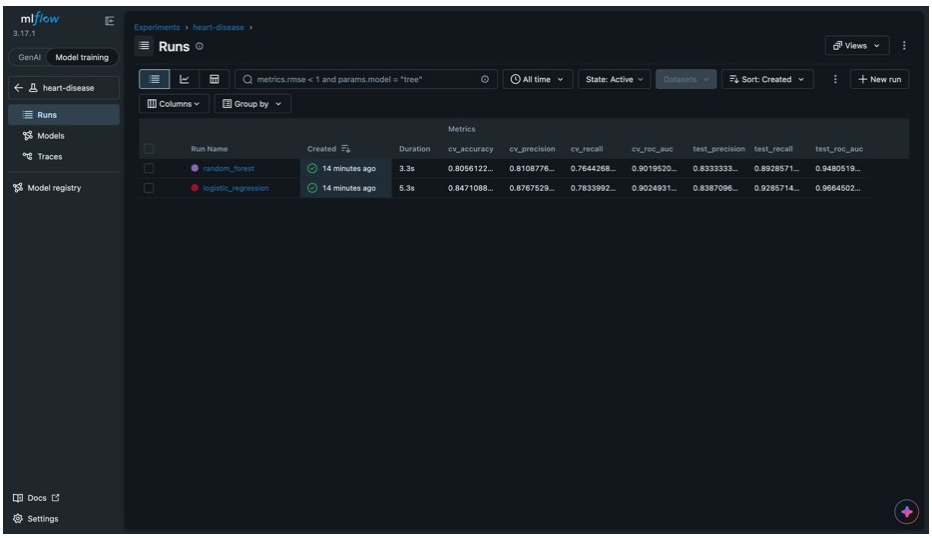


Figure 7. MLflow experiment view showing the tracked runs and metrics.

The fitted end-to-end pipeline is serialized in models/model.joblib. models/metadata.json records expected feature names, chosen parameters and evaluation scores. The Docker image installs the pinned runtime dependency file, while the development requirements cover data ingestion, training, notebook execution and tests.

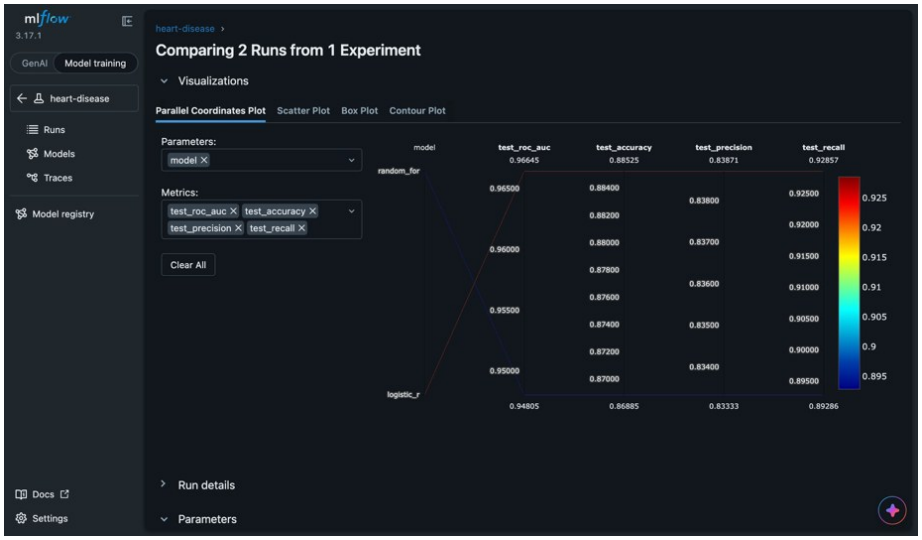


Figure 8. MLflow comparison between Logistic Regression and Random Forest.

Reproducibility controls

- Fixed random seed (42), stratified train/test split and stratified five-fold CV.
- CI re-downloads and compares the cleaned dataset, retrains twice, compares model metadata and archives a pip freeze environment snapshot.
- Re-running in a different environment produced a small Random Forest ROC-AUC variation (approximately 0.947 versus 0.948); exact library-version locking is therefore important.

Run locally: python -m src.train; inspect using mlflow ui --backend-store-uri sqlite:///mlflow.db.

4. Automated testing and CI/CD pipeline

The repository uses the main orchestrator (.github/workflows/MlOps_pipeline.yaml) to invoke five reusable workflows in sequence. Pull requests primarily execute model/test checks; the extended chain runs on the main branch or by manual dispatch. Production deployment requires approval. In completed GitHub Actions run #28, all five jobs finished successfully.

Stage	Validation / output
1 — ML pipeline	flake8; 22 pytest tests; EDA; training; MLflow; reproducibility comparison
2 — Quality / IaC	SonarQube quality gate and Checkov analysis of Dockerfile and Kubernetes manifests
3 — Build / supply chain	Docker build → Trivy HIGH/CRITICAL vulnerability gate → ECR push by commit SHA
4 — EKS deploy	Production review; Kubernetes apply; rollout and /health smoke test; rollback attempt
5 — OWASP ZAP	OpenAPI-based API scan plus baseline scan; high-risk security gate and reports

The source CI evidence records 22 passing tests and 98% line coverage (193 statements, four missed); tests exercise preprocessing, model persistence, API validation, the download step and training entry point. The quality workflow transfers coverage to SonarQube and checks infrastructure-as-code policies.

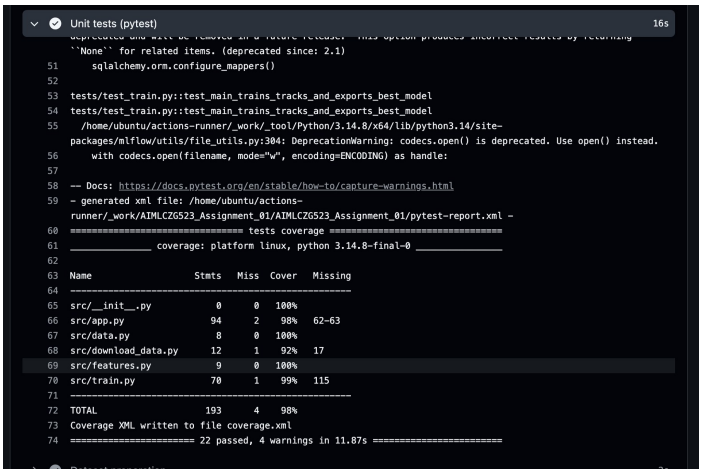


Figure 9. Recorded pytest and coverage output (22 tests; 98% line coverage).

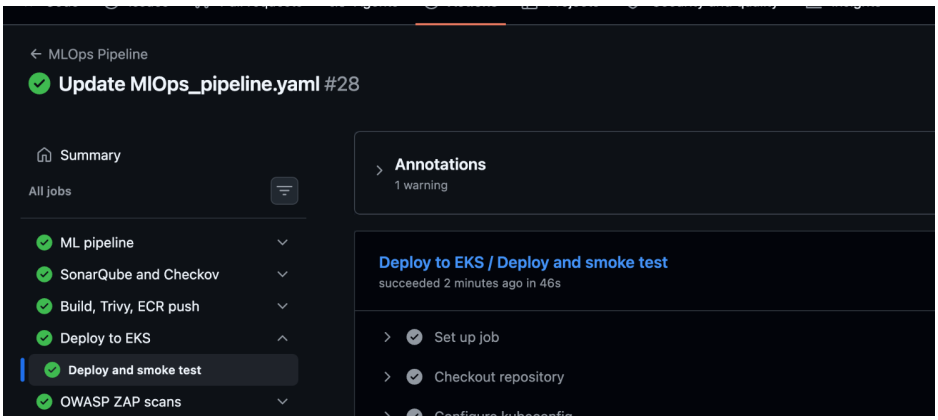


Figure 10. GitHub Actions workflow run #28: all five stages completed successfully.

5. Containerization, API and Trivy remediation

FastAPI loads the fitted pipeline at startup and validates inference requests through Pydantic. The Docker build uses a slim Python runtime, pinned serving dependencies, a non-root identity aligned with Kubernetes UID 10001, a health check and a reduced runtime footprint. Serving and training use matching Python/dependency environments to load model.joblib reliably.

Endpoint	Purpose
POST /predict	13 health-data fields; returns prediction, label, confidence and probability_disease
GET /health	Readiness/liveness response used by Docker and EKS
GET /metrics	Prometheus request, prediction and model-behaviour metrics
GET /docs	FastAPI-generated API reference and interactive schema

Local Docker build and inference validation

Local validation on macOS shows the image build for heart:latest completed successfully. A Docker container named heart-api was healthy and exposed TCP port 8000. A JSON POST request to /predict returned prediction 1 (heart disease), confidence 0.9856 and probability_disease 0.9856. The two terminal excerpts below capture the build, container state and sample inference response.

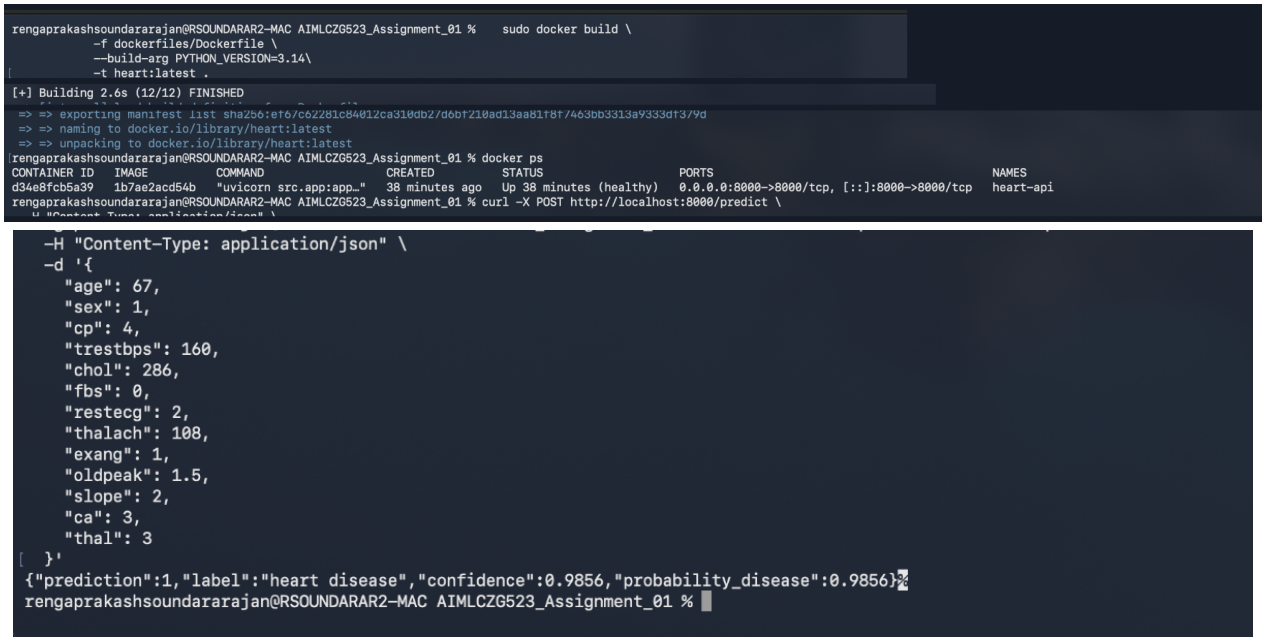


Figure 11. Local Docker smoke test on macOS: completed image build, healthy heart-api container and successful POST /predict with confidence 0.9856. Non-contiguous excerpts from one terminal capture; the running container predates this build.

Security-gate remediation

An earlier Trivy gate failed on four HIGH-severity findings and correctly blocked promotion. The findings were traced to vendored packaging components included with pip; the final image was rebuilt without unnecessary runtime packaging tools after verifying dependencies. The configured vulnerability gate remained enabled. Run #28 subsequently passed Trivy and pushed the verified commit-tagged image to Amazon ECR.

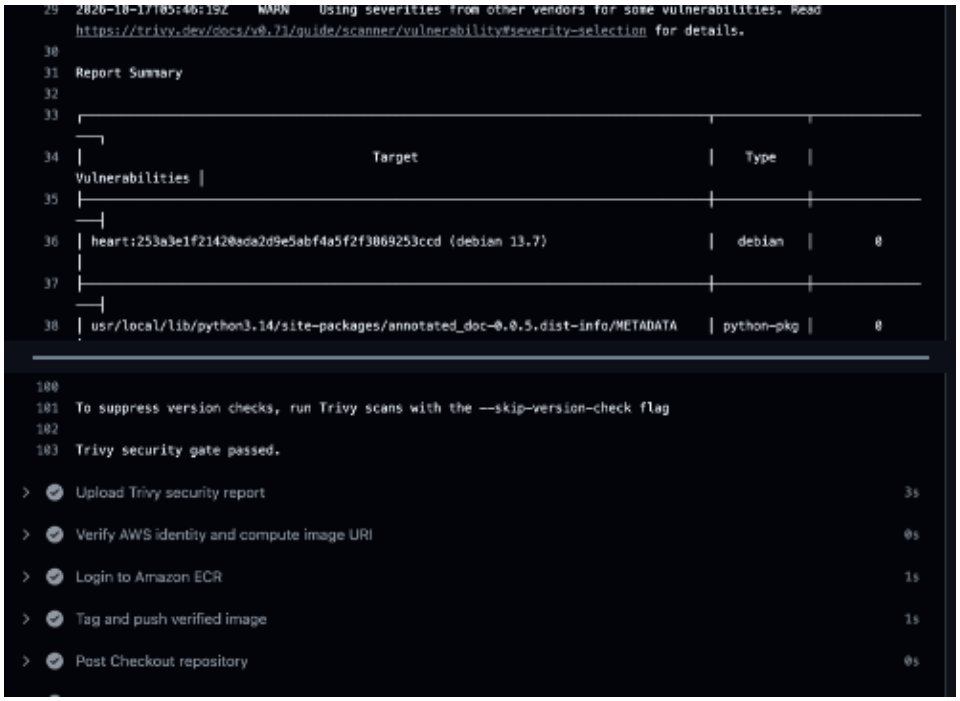


Figure 12. Run #28 Trivy log excerpts: zero findings in displayed inventory entries; security gate passed and image pushed to ECR. Excerpts are non-contiguous.

Final image evidence (run #28): Trivy scan PASS; vulnerability summary shows zero findings in its displayed OS/package targets; verified ECR push PASS. The scan used --ignore-unfixed, so the result establishes compliance with the configured gate, not a universal absence of vulnerabilities. Keep the full trivy-report artifact for audit.

For an authorized local test: docker build -f dockerfiles/Dockerfile -t heart:test .; docker run --rm -p 127.0.0.1:8000:8000 heart:test; curl http://127.0.0.1:8000/health.

6. AWS ECR and Amazon EKS deployment

After the passing Trivy scan, the approved container was pushed to the private heart-api ECR repository in ap-south-1, tagged with the source commit SHA. GitHub Actions run #28 then used the protected production environment to configure kubeconfig, apply manifests, wait for EKS rollout and perform a load-balancer smoke test. Every visible step in that deployment job passed.

The heartbeat-api namespace contains the heart-api Deployment, Service and NetworkPolicy. The manifests specify probes, CPU/memory limits, a non-root runtime, a read-only root filesystem, no privilege escalation and dropped Linux capabilities. The LoadBalancer serves HTTP on port 80 and forwards traffic to port 8000.

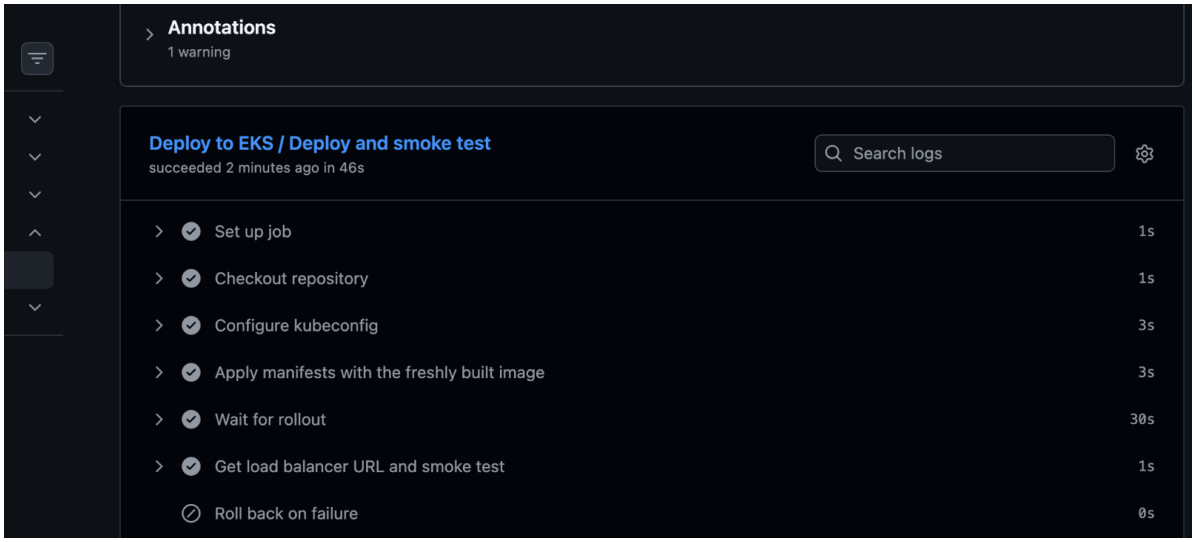


Figure 13. Run #28 EKS deploy job: manifests applied, rollout completed and load-balancer smoke test passed; rollback not needed.



Figure 14. Earlier endpoint demonstration: /health and /predict returned HTTP 200 via the load balancer.

Final deployment validation

The ten-point operator checklist was reported complete: cluster and node readiness, production approval, current desired replicas, deployed image reference, health and prediction responses, metrics, and saved evidence. Run #28 independently confirms a successful EKS rollout and load-balancer smoke test. Verify the final pod image digest against the Trivy-scanned ECR artifact in archived evidence.

Verification command	Acceptance
kubectl -n heartbeat-api rollout status deployment/heart-api	Successful rollout
kubectl -n heartbeat-api get deployments,pods,svc	Desired replicas Ready; ELB address assigned
kubectl -n heartbeat-api get deploy heart-api -o yaml	Correct ECR image reference, probes and security context

Deployment evidence: run #28 confirms successful manifest application, rollout and load-balancer health smoke test. Earlier source screenshots showed five replicas; the current manifest/checklist uses two. The external load-balancer address is temporary.

7. Monitoring and operational logging

The FastAPI middleware records HTTP method, route, response status and latency, while inference logging captures the predicted class and confidence. The /metrics endpoint exposes api_requests_total, api_request_latency_seconds and predictions_total, together with model-level metrics including test-performance gauges, inference timing, prediction probabilities and summaries of selected input fields.

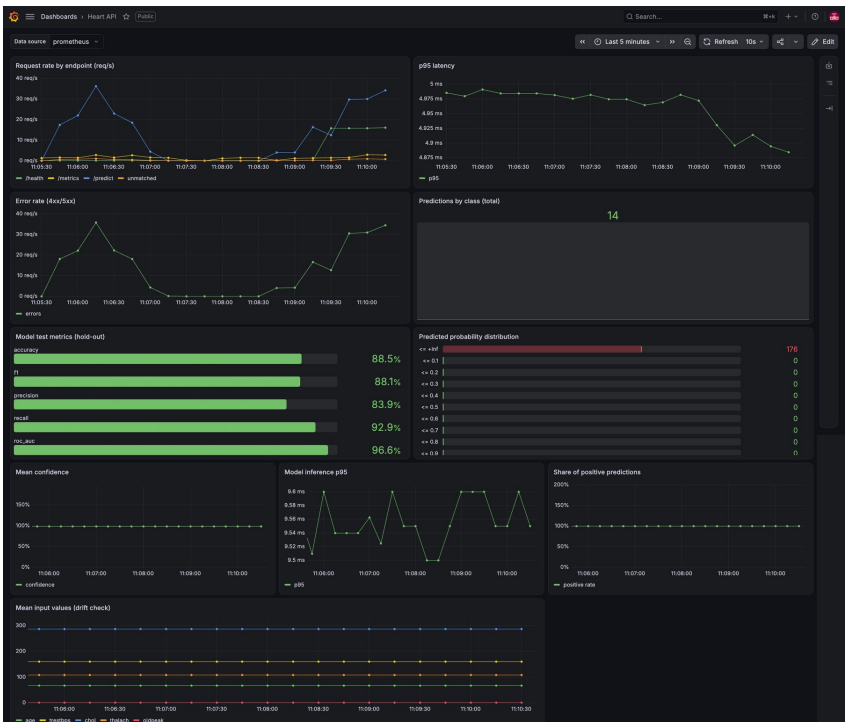


Figure 15. Grafana dashboard showing throughput, latency, model metrics and simple drift indicators.

The project runs Prometheus and Grafana on a separate VM; Prometheus polls the external service approximately every 15 seconds. The included Grafana dashboard queries request rates, p95 latency, 4xx/5xx errors and prediction counts.

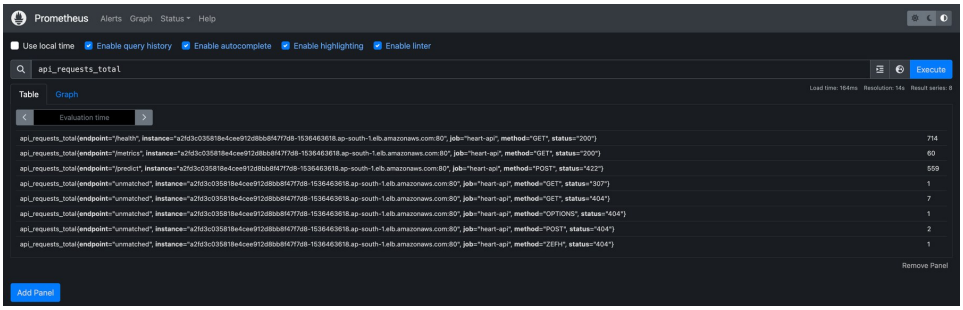


Figure 16. Prometheus scrape evidence for api_requests_total.

Reliability limitation

Scraping a multi-replica application through a single load-balancer target mixes counters from different pods and can distort rates. A production design should discover each pod as an independent scrape target through Kubernetes service discovery or a PodMonitor/ServiceMonitor, preserving the pod identity label. Current “drift” plots are descriptive input averages, not validated statistical drift detection.

8. Security validation and OWASP ZAP

Security verification combines SonarQube code quality, Checkov infrastructure scanning, the blocking Trivy container-image gate and OWASP ZAP API testing. GitHub Actions run #28 completed all five stages successfully. The archived final ZAP reports (ZAP 2.17.0; generated 17 October 2026) provide the finding-level evidence summarized below.

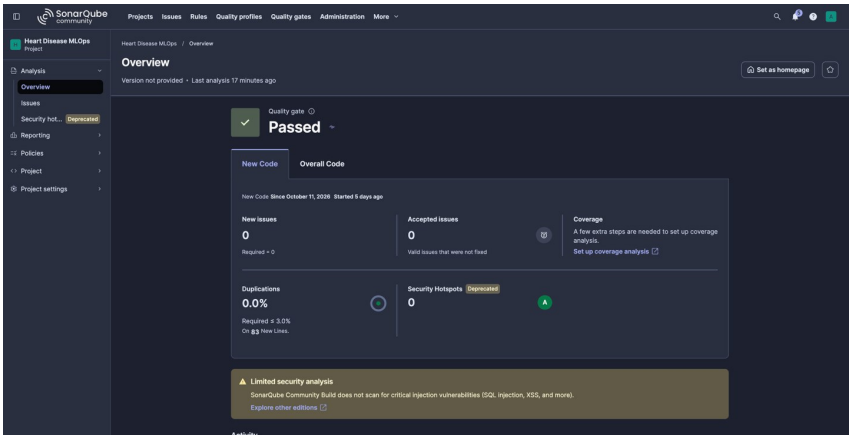


Figure 17. SonarQube quality gate shown as Passed in the original report.

ZAP findings and validation status

Final API scan: 0 HIGH, 0 MEDIUM, 2 LOW and 2 informational alerts (four distinct alert types). The LOW findings are missing Cross-Origin-Resource-Policy and X-Content-Type-Options headers, each observed on four routes; recommended remediation is an appropriate Cross-Origin-Resource-Policy (typically same-origin) and X-Content-Type-Options: nosniff. Informational findings concern unexpected-route/client errors and potentially cacheable responses; sensitive endpoints should return appropriate Cache-Control. The baseline scan found no HIGH, MEDIUM or LOW alerts and one informational cacheability alert, overlapping with the API scan. Its API scan insights reported 96% HTTP 4xx responses: invalid-input testing can cause these, but the rate merits review of scan coverage and valid-input cases.

Generated on Sat, 17 Oct 2026 05:58:14

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	2
Informational	2
False Positives:	0

Figure 18. Final OWASP ZAP API scan report, 17 Oct 2026: 0 HIGH, 0 MEDIUM, 2 LOW, 2 informational findings. Authentic report excerpt.

Scan coverage: the final API scan reported 96% HTTP 4xx responses. Retain the raw HTML/JSON findings; test valid request cases in addition to malformed inputs. Restrict active scans to authorized staging/demo systems with synthetic data. Review the LOW header findings despite the passing severity gate.

Residual risk: this academic demonstration exposes the API over HTTP without authentication or rate limiting. A green ZAP job does not establish comprehensive security; production readiness requires HTTPS, access control, safer monitoring exposure and validation of successful API test cases. Do not submit patient records.

9. Limitations, lessons learned and conclusion

Limitations and future improvements

- Clinical validity: 303 public records and 61 test patients produce uncertain estimates; this binary classifier is not suitable for diagnosis or clinical decision-making without external validation and calibrated probabilities.
- Exposure: public HTTP, unauthenticated /predict and /metrics endpoints, and no rate limiting. Add HTTPS with ACM, authentication, least-privilege access and request throttling.
- MLOps maturity: version-lock development dependencies, adopt immutable image-digest deployments, add statistical drift detection/alerting and use per-pod metrics scraping.
- Evidence retention: archive run #28 workflow results, Trivy scan/ECR image, EKS rollout and smoke test, plus the verified ZAP HTML/JSON files. ZAP follow-up: address both missing security headers, assess caching policy and review the API scan's 96% HTTP 4xx rate before claiming complete endpoint coverage.

Conclusion

The completed academic MLOps workflow demonstrates reproducible data preparation, leakage-aware modelling, MLflow tracking, automated tests, image security gating, approved EKS deployment and operational monitoring. Logistic Regression achieved 88.5% test accuracy, 92.9% recall and 0.966 ROC-AUC on 61 test records. GitHub Actions run #28 completed all five stages; the final uploaded ZAP reports confirm 0 HIGH and 0 MEDIUM API-scan alerts, with two LOW security-header findings still requiring remediation. Add the demonstration-video link before submission.

Submission links and evidence index

Deliverable	Location / final check
Source repository	https://github.com/Rengaprakash4440/AIMLCZG523_Assignment_01
Dataset, notebook, source and tests	data/ · report/notebooks/ · src/ · tests/
Tracked model and CI evidence	models/ · GitHub Actions artifacts · screenshots/
Deployment / monitoring	kubernetes/ · monitoring/ · screenshots/
Written report	This expanded report; PDF export
Demonstration video	[INSERT VERIFIED SHAREABLE VIDEO URL]
Final ZAP evidence	Final ZAP API scan (17 Oct 2026): 0 HIGH / 0 MEDIUM / 2 LOW / 2 INFO; HTML and JSON evidence supplied

References: UCI Machine Learning Repository, Heart Disease dataset (ID 45); project README and repository sources; recorded CI/MLflow/security outputs and the original assignment report.

Pre-submission checks: add a shareable video URL, confirm the submission date and run #28 warning annotation, verify image SHA/digest alignment, and archive ZAP report artifacts; security-header hardening and further valid-input ZAP coverage are documented follow-up improvements.

Appendix A. Setup, installation and reproducibility

This appendix provides a reproducible setup path for the submitted heart-disease MLOps project. Execute commands from the repository root unless stated otherwise. Start with the local workflow; cloud steps are required only to reproduce the AWS deployment and CI/CD evidence.

Reproducibility scope: use public UCI data or synthetic API requests only. The academic model is not clinically validated; the demonstration endpoint is not suitable for personal health records.

A.1 Prerequisites and repository components

Environment	Software / access needed
Local development	Git, 64-bit Python 3.14, pip and a terminal. Use the Python version matching the Docker build to avoid joblib/scikit-learn serialization mismatches.
Local container test	Docker Engine or Docker Desktop, with a running Docker daemon. Trivy is optional locally; it is mandatory in the configured CI security gate.
AWS deployment (optional)	AWS account with authorized ECR/EKS access; AWS CLI v2, kubectl, and eksctl when creating a new cluster. Region: ap-south-1.
Automated CI/CD	GitHub Actions, a self-hosted Linux runner with Docker/Trivy/Checkov/AWS CLI/kubectl, SonarQube access and protected production approval.
Observability	Prometheus and Grafana on the monitoring VM, or equivalent authorized monitoring services.

Preflight (run tools applicable to your environment)

```
git --version
python --version
docker version
# For AWS work: aws --version; kubectl version --client; eksctl version
```

The project has unpinned development dependencies in requirements.txt; record a pip freeze for repeatability. The serving image uses requirements-api.txt and Python 3.14. Training and serving libraries must remain compatible.

A.2 Clone source and create the Python environment

Linux / macOS / WSL (Bash)

```
git clone https://github.com/Rengaprakash4440/AIMLCZG523_Assignment_01.git
cd AIMLCZG523_Assignment_01
python3.14 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Windows PowerShell (alternative to the Bash steps)

```
git clone https://github.com/Rengaprakash4440/AIMLCZG523_Assignment_01.git
cd AIMLCZG523_Assignment_01
py -3.14 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

If PowerShell blocks virtual-environment activation, run .\.venv\Scripts\python.exe directly instead of changing machine-wide execution policy. Verify the interpreter with python --version, then record the installed versions and Git commit:

```
python -m pip freeze > environment-freeze.txt
git rev-parse HEAD
```

A.3 Download, clean and explore the UCI dataset

The download script retrieves UCI Heart Disease dataset ID 45 into data/heart_raw.csv. Cleaning binarizes the target (0 = no disease; 1-4 = disease), removes duplicate rows, and writes data/heart_clean.csv. Preprocessing imputation is deferred to model training.

```
python -m src.download_data
python -c "from src.data import load_clean; load_clean().to_csv('data/heart_clean.csv', index=False)"
python -c "import pandas as pd; print(pd.read_csv('data/heart_clean.csv').shape)"
```

The recorded data contains 303 rows, 13 predictors and one binary target (164 negative and 139 positive). Open the EDA notebook to reproduce the visualizations:

```
jupyter notebook report/notebooks/01_eda.ipynb
```

Notebook figures are found under report/figures/. Network access is required for a fresh UCI download; a committed cleaned CSV is also included in the repository.

A.4 Run automated tests, train and inspect MLflow

The model workflow tests the code, trains Logistic Regression and Random Forest with five-fold stratified cross-validation, and saves the selected fitted preprocessing/classifier pipeline. Run the checks from the project root:

```
flake8 src tests
pytest -v
python -m src.train
```

Expected artifacts include models/model.joblib, models/metadata.json, grid-search CSV files, ROC/confusion-matrix plots, and the SQLite MLflow database mlflow.db. View the recorded runs from a second terminal with the same environment activated:

```
mlflow ui --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port 5000
```

Open http://127.0.0.1:5000 and compare the two runs in the heart-disease experiment. The report records the selected Logistic Regression model and held-out evaluation; results may change if dependency versions differ.

A.5 Launch and validate the FastAPI service locally

Start the API after verifying models/model.joblib exists. Run the server bound to the loopback interface to prevent unnecessary exposure outside the development machine:

```
uvicorn src.app:app --host 127.0.0.1 --port 8000
```

Verify from another terminal (Bash; PowerShell use curl.exe)

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/metrics
```

The health endpoint should return {"status":"ok"}. Browse http://127.0.0.1:8000/docs for the OpenAPI interface. For a sample prediction, create sample-patient.json in the repository root with the following synthetic request:

```
{
  "age":67, "sex":1, "cp":4, "trestbps":160, "chol":286,
  "fbs":0, "restecg":2, "thalach":108, "exang":1,
  "oldpeak":1.5, "slope":2, "ca":3, "thal":3
}
```

Prediction request (Bash / macOS / WSL)

```
curl -fsS -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  --data-binary @sample-patient.json
```

Windows PowerShell prediction request

```
Invoke-RestMethod -Uri http://127.0.0.1:8000/predict `
  -Method Post -ContentType application/json -InFile .\sample-patient.json
```

The response includes prediction (0 or 1), label, confidence and probability_disease. Confidence is a predicted-class probability, not a clinically calibrated risk. Invalid or missing required fields should produce HTTP 422.

A.6 Build and test the Docker container

Stop the local Uvicorn process before binding Docker to the same port. Docker packages the API and trained model on Python 3.14, then serves it using a non-root identity and container health checks. Multiline blocks below use Bash; for PowerShell, join continuations on one line or use its backtick continuation character.

```
docker build -f dockerfiles/Dockerfile \
  --build-arg PYTHON_VERSION=3.14 -t heart-api:local .
docker run --rm -d --name heart-api-demo \
  -p 127.0.0.1:8000:8000 heart-api:local
docker ps
curl -fsS http://127.0.0.1:8000/health
docker logs heart-api-demo
```

After the health probe becomes healthy, issue the same POST /predict request as above. On completion, stop the local container with docker stop heart-api-demo. A local Trivy check can apply the repository's configured HIGH/CRITICAL policy:

```
trivy image --scanners vuln --severity HIGH,CRITICAL \
  --ignore-unfixed --exit-code 1 heart-api:local
```

A passing Trivy gate only means no blocking findings were detected under this policy; --ignore-unfixed excludes vulnerabilities without available fixes. Preserve the complete scan log.

A.7 Configure and execute GitHub Actions CI/CD

The main workflow is .github/workflows/MlOps_pipeline.yaml (filename capitalization matters). A push to main or a manual Actions run executes the five dependent stages. Pull requests execute the ML stage only.

Stage	Key verification
1. ML pipeline	Lint, 22 pytest checks in the recorded run, EDA, training, MLflow and reproducibility artifacts.
2. SonarQube / Checkov	SonarQube quality gate and Docker/Kubernetes infrastructure policy checks.
3. Build / Trivy / ECR	Docker image built, blocking vulnerability scan completed, commit-SHA image pushed to heart-api ECR.
4. EKS deployment	Production approval, Kubernetes apply, rollout wait, load-balancer /health smoke test and rollback handler.
5. OWASP ZAP	Authorized OpenAPI scan and baseline scan; JSON/HTML security reports uploaded as artifacts.

One-time GitHub / AWS preparation

- Set repository variable SONAR_SECRET_ARN to the approved AWS Secrets Manager secret for SonarQube; keep actual secret values out of the repository.
- Register an authorized self-hosted runner with Docker, Trivy, Checkov, AWS CLI and kubectl. Prefer its IAM instance profile and an EKS access entry; do not store long-lived AWS keys in workflow files.
- In GitHub Settings > Environments, configure production with a required reviewer. Confirm the ECR repository, EKS cluster and the runner role permissions before deploying.

Open the repository Actions tab, choose MLOps Pipeline and select Run workflow (or review a successful main-branch push). Confirm each stage is green and download its artifacts. The submitted evidence documents run #28. Preserve the run ID and commit SHA used by the image.

A.8 Deploy or verify the API in Amazon EKS

Preferred route: use the approved GitHub Actions workflow above because it enforces the quality, Trivy, ECR and production-review gates. The following terminal commands validate a deployment already created by that workflow:

EKS verification (AWS account and kubectl access required)

```
aws sts get-caller-identity
aws eks update-kubeconfig --name heart-mlops --region ap-south-1
kubectl get nodes
kubectl -n heartbeat-api rollout status deployment/heart-api
kubectl -n heartbeat-api get deployments,pods,svc -o wide
kubectl -n heartbeat-api get deploy heart-api -o yaml
```

Check that the desired number of replicas is Ready (the current manifest specifies two); the service receives a LoadBalancer hostname; and the pod template references the promoted ECR image. Check pod image IDs against the image verified by the Trivy gate.

Retrieve the temporary load balancer hostname

```
kubectl -n heartbeat-api get svc heart-api \
  -o jsonpath="{.status.loadBalancer.ingress[0].hostname}"
```

From an authorized client, open http://<ELB-HOSTNAME>/health and http://<ELB-HOSTNAME>/docs, and test POST /predict using the same synthetic payload. Wait for DNS to propagate if a newly created load balancer is not yet resolvable.

Optional: create a fresh test cluster only when authorized

```
eksctl create cluster --name heart-mlops --region ap-south-1 \
  --nodegroup-name workers --node-type c7i-flex.large \
  --nodes 1 --managed
```

Cluster creation incurs AWS charges and depends on quota and instance-type availability. Ensure the ECR repository and a successfully scanned image exist before invoking the deployment workflow. Do not provision a duplicate cluster if the assignment cluster is already running.

A.9 Manual ECR image publishing (fallback procedure)

Use this Bash procedure only when CI publishing is unavailable. It preserves the essential order: local image build, blocking Trivy scan, then ECR push. An IAM principal with appropriate ECR permissions and an existing repository named heart-api are required.

```
export AWS_REGION=ap-south-1
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
ECR="$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com"
SHA=$(git rev-parse HEAD)
docker buildx build --platform linux/amd64 \
  -f dockerfiles/Dockerfile -t heart-api:validated --load .
trivy image --scanners vuln --severity HIGH,CRITICAL \
  --ignore-unfixed --exit-code 1 heart-api:validated
aws ecr get-login-password --region "$AWS_REGION" | \
  docker login --username AWS --password-stdin "$ECR"
docker tag heart-api:validated "$ECR/heart-api:$SHA"
docker push "$ECR/heart-api:$SHA"
```

Before the push, create the ECR repository if missing with aws ecr create-repository --repository-name heart-api --region ap-south-1. After a successful push, resolve its immutable digest and apply only the verified image:

```
DIGEST=$(aws ecr describe-images --region "$AWS_REGION" \
  --repository-name heart-api --image-ids imageTag="$SHA" \
  --query "imageDetails[0].imageDigest" --output text)
IMAGE="$ECR/heart-api@$DIGEST"
kubectl apply -f kubernetes/namespaces.yaml
sed "s|image: .*|image: $IMAGE|" kubernetes/deployment.yaml | \
  kubectl apply -f -
kubectl apply -f kubernetes/service.yaml
kubectl apply -f kubernetes/network-policy.yaml
kubectl -n heartbeat-api rollout status deployment/heart-api
```

Verify the deployed digest and the load-balancer health response after rollout. Do not promote unscanned images. If no cluster exists, use the optional provisioning step in A.8.

A.10 Configure Prometheus, Grafana and application logs

The recorded project uses an external monitoring VM. Configure Prometheus to scrape GET /metrics at approximately 15-second intervals. Replace <ELB-HOSTNAME> with the hostname obtained above:

```
scrape_configs:
  - job_name: heart-api
    metrics_path: /metrics
    scrape_interval: 15s
    static_configs:
      - targets: ["<ELB-HOSTNAME>:80"]
```

Restart or reload Prometheus using the monitoring VM's service controls, then confirm target status UP and query api_requests_total. In Grafana, add Prometheus as a data source and import monitoring/heart-api-dashboard.json. Validate request throughput, p95 latency, HTTP errors and prediction counters; view logs using:

```
kubectl -n heartbeat-api logs -l app=heart-api --tail=100
# For a local container instead: docker logs heart-api-demo
```

Reliability warning: scraping an external LoadBalancer in front of two pods can interleave different counter streams. Production monitoring should scrape per-pod targets using Kubernetes discovery/ServiceMonitor/PodMonitor and restrict external access to /metrics.

A.11 Security verification, evidence and cleanup

On the Actions run page, download the Trivy artifact and OWASP ZAP HTML/JSON reports. The final documented ZAP API scan recorded 0 HIGH, 0 MEDIUM, 2 LOW and 2 INFO alert types; the 96% HTTP 4xx rate requires successful-request coverage review. Authorized staging/demo endpoints and synthetic data only.

Recommended evidence checklist

- Local installation: Python version, pip freeze, Git commit, data-cleaning output and test pass summary.
- Model reproduction: MLflow run comparison, models/metadata.json, saved model and plots.
- Container and security: local Docker /health and /predict, Trivy log, ECR image URI and verified digest.
- Cloud: run #28 workflow summary, production approval, EKS pods/service, rollout and API response.
- Observability and ZAP: Prometheus/Grafana views, API logs, ZAP HTML/JSON and documented LOW findings.

If the test cluster and repository were created solely for this exercise, remove them after saving evidence and confirming no shared workloads depend on them. This procedure is destructive and requires explicit resource ownership verification:

```
eksctl delete cluster --name heart-mlops --region ap-south-1
aws ecr delete-repository --repository-name heart-api \
  --force --region ap-south-1
```

Check the AWS console for orphaned load balancers and node groups. GitHub Actions artifacts can expire (the current ZAP workflow specifies seven-day retention), so archive the final reports promptly. Additional troubleshooting guidance and project source paths are available in the repository README.